Employee Salary Advance — User Guide

Technical name: `odomate_hr_salary_advance` · Version `19.0.1.0.4` · Odoo 19

Table of contents

1. What this module does

An employee asks HR for money before payday. This module turns that conversation into a tracked record:

1. HR records the request against the employee.
2. The request is checked against a company policy (a percentage of the monthly salary, an optional absolute ceiling, and whether more than one open advance is allowed at a time).
3. On approval, HR disburses it through a **real** `account.payment` — an outbound supplier payment posted to a bank or cash journal.
4. The full outstanding balance is pushed onto the employee's next payslip as an `ADV_DEDUCT` input line and written back as recovered when that payslip is confirmed. When the balance reaches zero, the advance closes by itself.

One model does the work: `odomate.hr.salary.advance`. Two short wizards (`odomate.hr.salary.advance.refuse` and `odomate.hr.salary.advance.pay`) handle refusal and disbursement.

2. Installation

Dependencies — `hr`, `payroll` (OCA), `account`, `mail` — are installed automatically if they are not present yet.

1. **Apps → Update Apps List**.
2. Search for **Employee Salary Advance** and click **Install**.

At install time the module:

- creates the `ADV/` sequence (`ADV/00001`, `ADV/00002`, …);
- ships the `ADV_DEDUCT` salary rule and its matching rule input;
- resolves the rule's category by searching `hr.salary.rule.category` for code `DED`, creating that category only if the database genuinely has none. (This is why the module never references `payroll.DED` — that xmlid exists only in the payroll module's demo data and is absent from most production databases.)

On a database with the Ukrainian language installed, error messages, button labels and window titles raised from the module's Python code — not just its views — now render in Ukrainian. Every Python-sourced entry in `i18n/uk.po` carries the `#. odoo-python` marker Odoo's loader requires before it will load a code translation, and the module no longer ships a `.pot` file that would otherwise overwrite that marker at load time.

---

3. Configuration

Two things must be done before the first advance can be paid.

3.1 Company policy

Settings → Employees → Salary Advances

Setting	Field	Default	Meaning
Max Advance (% of Monthly Salary)	`advance_max_percent`	`50`	Share of the monthly wage that can be advanced.
Max Advance Amount	`advance_max_amount`	`0`	Absolute ceiling on top of the percentage. `0` means no hard ceiling.
Allow Multiple Open Advances	`advance_allow_multiple`	off	Whether an employee may hold a second advance while one is still being recovered.
Salary Advance Journal	`advance_journal_id`	(empty)	Bank or cash journal used to disburse. **Required** — the Pay wizard refuses until it is set.

These live on `res.company`, so each company in a multi-company database has its own policy.

3.2 Attach the deduction rule to your salary structures

The module ships the `ADV_DEDUCT` salary rule but **deliberately does not attach it to any salary structure**. Which structures should carry the deduction is your decision, and silently editing a

customer's payroll structures is not something a module should do.

Do this once, per structure that should recover advances:

1. **Payroll → Configuration → Salary Structures**.
2. Open the structure (e.g. Base for new structures).
3. In **Salary Rules**, add **Salary Advance Deduction** (code `ADV_DEDUCT`).

Until this is done, the deduction line is generated on the payslip's **Other Inputs** tab but produces no payslip line and nothing is recovered.

The rule itself is:

- **Condition** (Python): sums the `amount` of every `ADV_DEDUCT` line on `payslip.input_line_ids` and is true when that sum is non-zero
- **Amount** (Python): the negative of that same sum
- **Category**: the `DED` category resolved at install

(Both expressions read directly from `payslip.input_line_ids` rather than from the payroll engine's `inputs` helper object — the latter raises on a plain membership test in this OCA payroll version, which would otherwise break every payslip on any structure carrying this rule.)

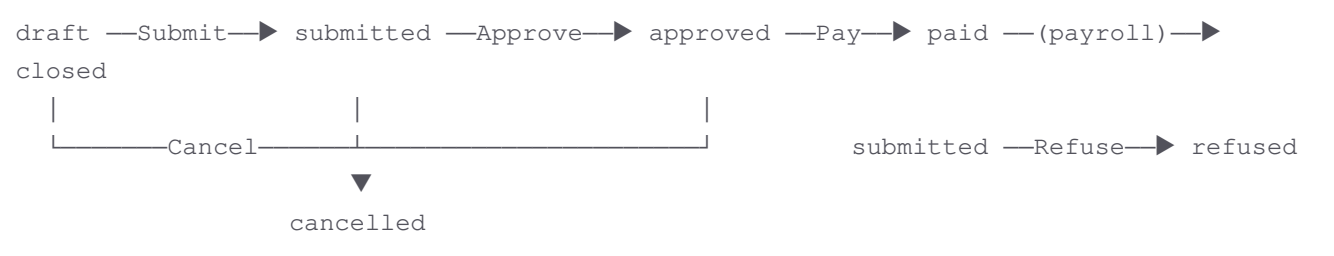
The code `ADV_DEDUCT` is intentionally different from the loan module's `LOAN_REPAY`, so a payslip carrying both keeps them as separate, separately-auditable lines.

---

4. Daily use

Employees → Salary Advances → Salary Advances

4.1 The lifecycle



Stage	Buttons available
Draft	Submit, Cancel
Submitted	Approve, Refuse, Cancel
Approved	Pay, Cancel
Paid	(none — recovery is automatic)

Closed	(none)
Refused	(none)
Cancelled	(none)

Refused and cancelled are dead ends — there is no "Set to Draft". This is deliberate. A refusal's written reason belongs on the permanent record rather than being erased by reopening it, and an advance that "never happened" is superseded by simply raising a new request.

4.2 Recording a request

1. Click **New**.
2. Pick the **Employee**. The reference (`ADV/00001`) is assigned on save.
3. **Request Date** defaults to today. It is also the date the wage lookup resolves against — backdate it and the module reads the contract that was in force then, not today's.
4. Enter the **Advance Amount** and a **Reason** (both required).
5. Watch the **Eligibility** block — it is live and computed before you even save.
6. Click **Submit**, then **Approve**.

Both Submit and Approve re-run the same three checks, and the checks are enforced in the model (an `@api.constrains`), not just behind the buttons — so an import or an RPC call cannot bypass them:

- the amount must not exceed **Available**;
- the employee must not already hold an open advance (unless Allow Multiple Open Advances is on);
- a contract (`hr.version`) must be in force on the request date.

4.3 Refusing

Refuse (available on a submitted advance) opens a small wizard that requires a reason. The reason is stored on `refusal_reason`, posted to the chatter, and shown in the **Refusal** block on the form.

4.4 Paying

Pay (available on an approved advance) opens a wizard with a payment date and a journal, pre-filled from the company's Salary Advance Journal.

Confirming does all of this in one transaction:

1. Refuses outright if the company has no **Salary Advance Journal** configured — the module names the requirement instead of guessing a journal.
2. Refuses outright if the employee has no **work contact** (`work_contact_id`) — the payment needs a partner and the module will not invent one or substitute somebody else's.

3. Otherwise creates an `account.payment` (`payment_type = outbound`, `partner_type = supplier`, partner = the employee's work contact) and posts it.
4. Only then writes `payment_id`, `payment_date`, `journal_id` and `state = paid`.

If posting fails, the whole transaction rolls back: the advance stays **Approved** and no orphaned draft payment is left behind.

Once paid, two smart buttons appear — **Payment** and **Journal Entry**. They are not elevated: a user without accounting rights gets an access error rather than a view of the books.

---

5. How the eligibility figures are calculated

Four figures sit in the **Eligibility** block, all recomputed live.

Figure	Formula
Monthly Salary	`wage` of the `hr.version` in force on the request date
Maximum Allowed	`min(Monthly Salary × Max % ÷ 100, Max Amount)` — the ceiling is skipped when it is `0`
Already Outstanding	sum of `outstanding_amount` over this employee's other advances in **Approved** or **Paid**
Available	`max(Maximum Allowed − Already Outstanding, 0)`

Worked example

Olena earns **3 000.00** a month. The company allows **50 %** with no absolute ceiling, and she already has one advance of **1 000.00** that has been paid but not yet recovered.

```
Monthly Salary       = 3 000.00
Maximum Allowed      = 3 000.00 × 50 / 100 = 1 500.00
Already Outstanding  =                       1 000.00
Available            = 1 500.00 − 1 000.00 =   500.00
```

A request for 400.00 passes the amount check. A request for 600.00 is refused with: "The requested amount exceeds what Olena Kravets may take. Maximum allowed: 1500.00; already outstanding: 1000.00; still available: 500.00."

Note that with Allow Multiple Open Advances off — the default — the second request is refused before the amount is even considered, because one advance is already open.

Which contract is read

The module takes the latest `hr.version` whose `date_version` is on or before the request date, and drops it if its own `date_end` has already passed. Because **Request Date** is required and defaults to today, this coincides with the employee's current version in the ordinary case; it only diverges when HR deliberately backdates a request. The "no active contract" check uses that same resolved date,

so the wage and the contract check can never disagree.

6. Payroll recovery

6.1 What lands on the payslip

When a payslip is built for an employee — whether through the form (which recomputes the deduction as soon as employee/dates change) or through a batch payroll run (**Compute Sheet**) — the module appends one input line per recoverable advance:

- code `ADV_DEDUCT`
- description **Salary Advance Deduction**
- amount = the advance's current **Outstanding** figure
- linked back to the advance through `hr.payslip.input.advance_id`

Only advances in state **Paid** contribute, and only to a payslip whose period ends on or after the disbursement date. Approved-but-unpaid, refused, cancelled and closed advances contribute nothing.

Recomputing a draft or to-verify payslip more than once (e.g. running **Compute Sheet** again) replaces this module's own lines rather than duplicating them — any other input line you or another module added by hand is left untouched.

The rule computes the negative of the summed `ADV_DEDUCT` line amounts, so it lands as a negative (deduction) line among the other deductions.

6.2 What happens on confirmation

When the payslip is confirmed (**Confirm**, `action_payslip_done`), the module reads back **the confirmed input line's own amount** — not the advance's stored figure. If a payroll officer edited the line down from 600.00 to 200.00 because the employee asked to spread it, 200.00 is what gets recovered.

That amount is added to **Recovered**, **Outstanding** drops accordingly, and the advance moves to **Closed** only when the balance reaches zero.

Recovery is idempotent and never over-recovers:

- only payslips actually transitioning into `done` are counted, so re-confirming an already-confirmed payslip recovers nothing;
- **Recovered** is clamped at the advanced amount.

6.3 Worked example

A 900.00 advance paid on 3 March, with the deduction taken over two months:

Event	Input line	Recovered	Outstanding	State

Paid 3 March	—	0.00	900.00	Paid
March payslip generated	900.00	0.00	900.00	Paid
Officer edits line to 400.00, confirms	400.00	400.00	500.00	Paid
April payslip generated	500.00	400.00	500.00	Paid
April payslip confirmed	500.00	900.00	0.00	**Closed**

7. Roles and access rights

No new security group is created — the module reuses the standard HR ladder, and it grants that ladder to nobody. Existing HR officers and administrators simply gain the new screens.

Group	Advance records	Workflow actions	Config screen
`hr.group_hr_user` (Officer)	read, write, create	Submit, Approve, Refuse, Cancel, Pay	no
`hr.group_hr_manager` (Administrator)	read, write, create, **delete**	all of the above	yes
`base.group_user` (any internal user)	**read only, own records**	none	no

Record rules on `odomate.hr.salary.advance`:

- a **global** multi-company rule — `company_id` must be in the user's allowed companies, applied to everyone including HR;
- HR officers see every request in their allowed companies;
- every other internal user reads only records where `employee_id.user_id = self`, which is what **My Advances** shows.

Two further guarantees:

- The workflow methods check the HR group **themselves** (`_check_hr_officer`), so hiding a button is not the only thing standing between an employee and an approval.
- HR gains **no** `account.*` rights. The Pay wizard's `sudo()` is scoped to the single create-and-post call and is never carried across the method, so an HR officer without an accounting role can pay an advance and still cannot open the resulting journal entry.

8. Field reference

`odomate.hr.salary.advance`

Field	Type	Notes
`name`	Char	`ADV/00001`, sequence-generated, unique
`employee_id`	Many2one `hr.employee`	required
`department_id`	Many2one `hr.department`	related, stored
`company_id`	Many2one `res.company`	required
`currency_id`	Many2one `res.currency`	company currency
`date`	Date	required, defaults today; drives the wage lookup
`amount`	Monetary	required, `> 0`, locked once paid
`reason`	Text	required
`state`	Selection	draft / submitted / approved / paid / closed / refused / cancelled
`refusal_reason`	Text	set only by the Refuse wizard
`payment_date`	Date	set only by the Pay wizard
`journal_id`	Many2one `account.journal`	set only by the Pay wizard
`payment_id`	Many2one `account.payment`	the journal entry is reached via `payment_id.move_id`
`recovered_amount`	Monetary	written only by payslip confirmation
`outstanding_amount`	Monetary	stored compute, `amount − recovered_amount`; forced to `0.00` when `state` is **Refused** or **Cancelled**, since nothing was ever disbursed and nothing is owed
`monthly_wage`	Monetary	computed, not stored
`max_allowed_amount`	Monetary	computed, not stored
`already_outstanding_amount`	Monetary	computed, not stored
`available_amount`	Monetary	computed, not stored

Additions to existing models

Model	Field	Notes

`hr.payslip.input`	`advance_id`	links the deduction back to its advance
`hr.employee`	`advance_count`	smart button — advances in Approved or Paid
`res.company`	`advance_max_percent`, `advance_max_amount`, `advance_allow_multiple`, `advance_journal_id`	policy, mirrored on `res.config.settings`

9. Limitations

Stated plainly, because knowing the edges is worth more than a longer feature list.

- **No instalment schedule.** The entire outstanding balance is pushed onto the next payslip. Spreading it over several months is possible, but only by editing the input line's amount by hand on each payslip (the module recovers whatever the confirmed line says). There is no repayment plan, no instalment records, and no interest.
- **No accounting for the recovery leg.** Disbursement creates and posts a real payment; the recovery stays a payroll concern and produces no journal entry of its own, and the two are never reconciled against each other.
- **The "one open advance" rule is enforced in Python, not by a database constraint.** It depends on a per-company setting that can be switched on and off, so it cannot be backed by a fixed unique index. Under genuinely simultaneous submissions of two advances for the same employee, both could pass the check. In practice HR submits these one at a time; if you need a hard guarantee, keep Allow Multiple Open Advances off and review the employee's advance count before approving.
- **No self-service.** Employees can read their own advances under **My Advances**, but only HR can create or progress one.
- **One approval step.** A single HR officer approves; there is no approval chain, no delegation and no escalation.
- **No automatic reminders**, no cron, and no notification when an advance sits unrecovered.
- **Departing employees are not flagged in the departure flow.** The employee form carries an **Advances** smart button showing how many advances are still open, and that is the only surfacing the module does — it is visible whether or not a departure is in progress, rather than something that activates because of one. Wiring a warning into `hr.departure.wizard` would need an extension point this module does not claim.
- **Single currency.** Amounts are always in the company currency; there is no second currency and no conversion.
- **No link to leave, attendance or worked days**, and no minimum service period before an employee becomes eligible.

- **Demo data posts no payments.** The four demo advances reach paid with `payment_id` empty, because posting into a demo company's books fails wherever accounting is not configured.

Employee Salary Advance — by OdoMate · support@odomate.pro